STRING PATTERN HANDBOOK

Pattern 1: Two Pointers (Palindrome Family)

Recognition Signals

If the problem contains:

- Palindrome
- Symmetric string
- Mirror property
- Compare both ends
- Remove one character and still valid
- Reverse string in-place

Think:

left = start;
right = end;

Move inward while comparing.

Core Invariant

Everything outside:

[left ... right]

has already been verified.

At every step:

s.charAt(left) == s.charAt(right)

must hold.

Generic Template

```
int left = 0;
int right = n - 1;

while(left < right){

   if(s.charAt(left) != s.charAt(right)){
      // mismatch handling
   }

   left++;
   right--;
}
```

Visualization

racecar

```
l          r
↓          ↓

r a c e c a r
```

move inward

```
  l   r
```

Mismatch Handling Variants

Type 1: Strict Palindrome

```
if(s[left] != s[right])
   return false;
```

Used in:

- Valid Palindrome

Type 2: One Deletion Allowed

```
if(s[left] != s[right]){

   return isPalindrome(left+1,right)
      || isPalindrome(left,right-1);
```

}

Used in:

- Valid Palindrome II

Type 3: Reverse String

swap(left,right);

Used in:

- Reverse String

Related Problems

Problem	Variant
Reverse String	Swap Ends
Valid Palindrome	Strict Match
Valid Palindrome II	Skip One Side
Longest Palindromic Substring	Center Expansion
Palindromic Substrings	Center Expansion Count

Interview Shortcut

If you see:

Palindrome
Mirror
Symmetric
Compare Ends

Think:

Two Pointers

If longest palindrome:

Center Expansion

Pattern 2: Sliding Window

Recognition Signals

Look for:

- Longest substring
- Shortest substring
- Maximum length
- Minimum window
- Continuous substring
- At most K
- Exactly K
- Unique characters
- Frequency constraints

Core Mental Model

Expand Right

Window becomes invalid?

Shrink Left

Restore validity

Continue

Visualization:

Valid
↓
Expand

Invalid
↓
Shrink

Valid

Universal Template

```
int left = 0;

for(int right=0; right<n; right++){

   add(right);

   while(windowInvalid()){

      remove(left);
      left++;
   }

   updateAnswer();
}
```

The 4 Questions Rule

For every Sliding Window problem:

Q1

What makes window VALID?

Q2

What makes window INVALID?

Q3

What changes during EXPANSION?

Q4

What changes during SHRINKING?

Pattern 3: Frequency Map Window

Most string problems are actually this pattern.

Recognition Signals

Keywords:

- Anagram
- Permutation
- Frequency
- Distinct
- Unique
- Count characters
- Minimum window

Think:

HashMap<Character,Integer>

or

int[] freq = new int[26];

Core Invariant

Map stores:

Current Window Frequency

NOT

Entire String Frequency

Subpattern A: Longest Valid Window

Recognition Signals

Longest
Maximum

Largest

with some condition.

Strategy

Expand aggressively

Shrink only when invalid

Example

Longest Substring Without Repeating Characters

Valid Window:

freq[ch] <= 1

Invalid:

freq[ch] > 1

Template:

```
while(freq[ch] > 1){
   remove(left);
   left++;
}
```

Answer:

ans = Math.max(ans,right-left+1);

Subpattern B: At Most K Distinct

Recognition Signals

K Distinct
K Unique

Window State

map.size()

Valid:

distinct <= K

Invalid:

distinct > K

Shrink until:

distinct <= K

Update answer when:

distinct == K

Example Problems

- Longest Substring with K Distinct
- Subarrays with K Different Integers

Subpattern C: Fixed Frequency Window

Recognition Signals

Anagram
Permutation
Rearrangement

Key Observation

Window size never changes.

windowSize = pattern.length()

Template

```
add(right);

if(windowSize > k){
  remove(left);
  left++;
}

checkMatch();
```

Example Problems

- Find All Anagrams
- Permutation in String

Subpattern D: Smallest Valid Window

Recognition Signals

Minimum
Smallest
Shortest
Containing All Characters

Example

Minimum Window Substring

Need:

Map<Character,Integer> need

and

required

Meaning:

How many characters
are still missing

Expand

need[ch]--;

if(need[ch] >= 0)
  required--;

Window Valid

required == 0

Shrink

need[left]++;

if(need[left] > 0)
  required++;

Strategy

Get Valid

Shrink Aggressively

Store Minimum

Master Classification Table

Pattern	Recognition
Two Pointer	Palindrome / Mirror
Center Expansion	Longest Palindrome
Longest Valid Window	Longest + Constraint
At Most K Distinct	K Unique
Fixed Window	Anagram / Permutation
Smallest Valid Window	Minimum + Contains All
Multi-Frequency Window	Concatenation of Words

10-Second Pattern Identification Cheat Sheet

Question says:

Palindrome

Use:

Two Pointer

Question says:

Longest Palindrome

Use:

Center Expansion

Question says:

Longest
Maximum
Largest

Use:

Expand Fast
Shrink When Invalid

Question says:

Minimum
Smallest
Shortest

Use:

Get Valid
Shrink Aggressively

Question says:

Anagram
Permutation

Use:

Fixed Frequency Window

Question says:

Distinct
Unique

Use:

HashMap Size

Mistake Log (Most Valuable Section)

Problem	Mistake
Longest Substring Without Repeating	Used if instead of while
Find All Anagrams	Used variable window
Minimum Window Substring	Updated answer before validity
Longest Palindromic Substring	Wrong start/end formula
Valid Palindrome II	Checked only one deletion path
Permutation in String	Forgot fixed-size window
K Distinct	Removed frequency but not map key

Missing Pattern (Add This)

You should add a separate section:

Pattern 4: Center Expansion

because:

- Longest Palindromic Substring

- Palindromic Substrings

are NOT true two-pointer problems.

They form their own pattern.

This is the biggest improvement I'd make to your notes.

Final rating of your current notes: **8.5/10**

After restructuring into **Pattern → Recognition → Invariant → Template → Variants → Problems → Mistakes**, they become **9.8/10 interview-ready notes** and much easier to revise before contests and interviews.

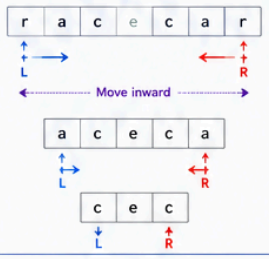
STRING PATTERNS: VISUAL PATTERNS & TEMPLATES

PATTERN 1 : TWO POINTER (PALINDROME FAMILY)

RECOGNITION SIGNALS

- Palindrome
- Symmetric string
- Compare both ends
- Remove one character and still valid
- Reverse string

VISUAL PATTERN



GENERIC TEMPLATE

```
int left = 0;
int right = n - 1;

while(left < right){
  if(s.charAt(left) != s.charAt(right)){
     // mismatch handling
  }
  left++;
  right--;
}
```

TYPES / VARIANTS

1. STRICT PALINDROME

```
if(s[left] != s[right])
    return false;
```
→ Valid Palindrome

2. ONE DELETION ALLOWED

```
if(s[left] != s[right]){
   return isPal(left+1, right) ||
          isPal(left, right-1);
}
```
→ Valid Palindrome II

3. REVERSE STRING

```
swap(s[left], s[right]);
```
→ Reverse String

PROBLEMS UNDER THIS PATTERN

- Reverse String
- Valid Palindrome
- Valid Palindrome II
- Longest Palindromic Substring (Center Expansion)
- Palindromic Substrings (Center Expansion Count)

SHORTCUT

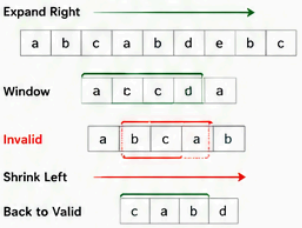
Palindrome / Mirror / Symmetric
=> Think Two Pointers
If longest => Center Expansion

PATTERN 2 : SLIDING WINDOW (VARIABLE WINDOW)

RECOGNITION SIGNALS

- Longest substring
- Shortest substring
- Maximum length
- Minimum window
- At most K / Exactly K
- Unique characters
- Frequency conditions

VISUAL PATTERN



UNIVERSAL TEMPLATE

```
int left = 0;
for(int right = 0; right < n; right++){
   add(right);            // expand

   while(invalid()){      // shrink
     remove(left);
     left++;
   }

   updateAnswer();
}
```

THE 4 QUESTIONS RULE

- **Q1** What makes window **VALID?**
- **Q2** What makes window **INVALID?**
- **Q3** What changes during **EXPANSION?**
- **Q4** What changes during **SHRINKING?**

SLIDING WINDOW CLASSIFICATION

Longest Valid Window	At Most K Distinct	Fixed Frequency Window	Smallest Valid Window	Multi Token Window
(Longest Substring Without Repeating)	(Longest K Unique)	(Anagram / Permutation)	(Minimum Window Substring)	(Concat. of All Words)
Goal: Maximize length of valid window Expand fast, shrink only when invalid	Goal: Max length with at most K unique characters	Goal: Match frequency in fixed window size	Goal: Smallest window that satisfies condition	Goal: Match multiple words with exact frequency

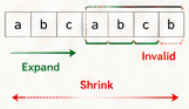
PATTERN 3 : FREQUENCY MAP WINDOW (SUBPATTERNS)

RECOGNITION SIGNALS: Anagram, Permutation, Frequency, Distinct, Unique, Count Characters, Minimum Window

CORE INVARIANT: Map stores CURRENT WINDOW FREQUENCY (Not entire string frequency)

A. LONGEST VALID WINDOW
(Longest Substring Without Repeating)

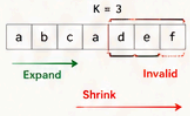
VALID: freq[ch] <= 1
INVALID: freq[ch] > 1




```
while(freq[ch] > 1){
  freq[s[left]]--;
  left++;
}
ans = max(ans, right-left+1);
```

B. AT MOST K DISTINCT
(Longest K Unique)

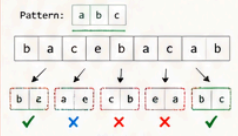
VALID: distinct <= K
INVALID: distinct > K




```
while(map.size() > K){
  remove(left);
  left++;
}
if(map.size() == K)
  ans = max(ans, right-left+1);
```

C. FIXED FREQUENCY WINDOW
(Find All Anagrams / Permutation in String)

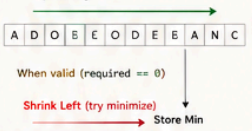
Window Size = Pattern Size (k)




```
add(right);
if(wIndowSize > k){
  remove(left);
  left++;
}
if(match())  storeIndex(left);
```

D. SMALLEST VALID WINDOW
(Minimum Window Substring)

Need Map + required




```
need[ch]--;
if(need[ch] >= 0) required--;

while(required == 0){
  updateAnswer();
  need[s[left]]++;
  if(need[s[left]] > 0) required++;
  left++;
}
```

E. MULTI TOKEN WINDOW
(Substring with Concatenation of All Words)

Words = [ab, cd, ef]
WordLen = 2, Count = 3

Use offset technique (0..wordLen-1)
+ HashMap of word frequency
+ Fixed window of size
wordLen * count

PROBLEM -> PATTERN MAP

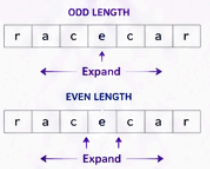
Longest Substring Without Repeating	Longest Substring with K Unique	Find All Anagrams	Permutation in String	Minimum Window Substring	Substring with Concatenation of All Words
A. Longest Valid Window	B. At Most K Distinct	C. Fixed Frequency Window	C. Fixed Frequency Window	D. Smallest Valid Window	E. Multi Token Window

PATTERN 4 : CENTER EXPANSION (PALINDROME CENTER FAMILY)

RECOGNITION SIGNALS

- Longest palindromic substring
- Count palindromic substrings
- Palindrome centered around an index

VISUAL PATTERN



TEMPLATE (Expand Around Center)

```
void expand(int left, int right){
  while(left >= 0 && right < n &&
      s.charAt(left) == s.charAt(right)){
    // update answer here
    left--;
    right++;
  }
}
for(int i = 0; i < n; i++){
   expand(i, i);     // odd
   expand(i, i+1);   // even
}
```

TYPES / USAGE

1. LONGEST PALINDROMIC SUBSTRING

Update start & length while expanding

2. PALINDROMIC SUBSTRINGS (COUNT)

For each valid expansion → count++

TIME COMPLEXITY

Center Expansion
$O(n^2)$

Two Pointer Palindrome Check
$O(n)$

10-SECOND PATTERN IDENTIFICATION

Palindrome / Mirror	→	Two Pointers / Center Expansion
Longest / Maximum / Largest	→	Expand Fast, Shrink When Invalid
Minimum / Smallest / Shortest	→	Get Valid, Shrink Aggressively
Anagram / Permutation	→	Fixed Frequency Window
Distinct / Unique	→	HashMap Size

COMMON MISTAKES LOG

Problem	Common Mistake	Problem	Common Mistake
1. Longest Substring Without Repeating	Forgot while-loop shrink	5. Valid Palindrome II	Checked only one deletion path
2. Find All Anagrams	Used variable window	6. Permutation in String	Forgot fixed-size window
3. Minimum Window Substring	Updated answer before validity	7. Longest K Unique	Removed frequency but not map key
4. Longest Palindromic Substring	Wrong start/end formula		